The loop is the whole agent

Animated explainer · src/agent.js

Everything else in src exists to serve this one function. It assembles a request, asks the model, dispatches the tools it asks for, and feeds every result back. This is the file that turns text into action.

9 scenes · 7 beats per step · 9 hook call sites · one exported function

CONTENTS

What is in this explainer

01 · The whole idea 3

02 · The contract 4

03 · The seven beats 5

04 · Messages are memory 6

05 · The loop in code 7

06 · Lifecycle 8

07 · Dispatch 9

08 · How it ends 10

09 · Recap 11

Page numbers refer to the folio at the foot of each page.

01 · The whole idea

One function. Seven beats.

SRC/AGENT.JS · *source*

```
export async function runTask({
  task, workspace, sandbox = null, commandPolicy, maxSteps, chatFn,
  hooks = null, memory = null, history = [], log = console.log,
}) {
  const messages = [
    { role: 'system', content: buildSystemPrompt({ memory, log }) },
    ...history.filter((m) => m.role !== 'system'),
    { role: 'user', content: task },
  ];
  // ...seven beats, once per step...
}
```

The model — One text-in, text-out request.

The tools — The model asks; the dispatcher acts.

The loop — Every observation goes back in.

> **On screen:** This is the whole agent: one exported function, no flags, no colours, no process concerns. Three things matter: a model that produces text, tools that do work, and a loop that feeds results back. Everything else in src/ serves this function. That is the shape of the whole program.

02 · The contract

What goes in.

task — The user request.

workspace — Where relative reads start, and what writes may not leave.

sandbox — The write fence. Created if not supplied.

commandPolicy — allow · ask · deny

maxSteps — How many model calls this run may make.

chatFn — The one model request. Injected, so tests can run offline.

hooks — The nine lifecycle moments. Optional.

memory — What survives this process. Optional.

history — Messages carried from the previous task in a session.

> Every dependency is injected. That is why the tests can run the real loop with a fake model and no network.
> **On screen:** Every dependency arrives as an option. The loop never reaches for a global. chatFn is the seam that makes the loop testable: a fake model can return a fixed tool call. The result is a small object: ok, a reason when it failed, the answer, messages, and the step count.

03 · The seven beats

One step, seven beats.

1. **Announce** — arm heartbeat
2. **Model call** — messages + tools
3. **Check truncation** — finish_reason
4. **Push assistant** — history
5. **No calls?** — answer + return
6. **Dispatch** — each tool
7. **Loop** — with observations

> A truncated reply is half a tool call, so it is never dispatched. The model is asked to continue smaller.
> **On screen:** The first three beats: announce the step, arm the heartbeat, and make one model call. Then push the assistant reply into history, check for tool calls, dispatch each one, and loop. No tool calls means the model is finished. Print the answer and return. That is the exit.

04 · Messages are memory

The conversation is the state.

SRC/AGENT.JS · *source*

```
const messages = [
  { role: 'system', content: buildSystemPrompt({ memory, log }) },
  ...history.filter((m) => m.role !== 'system'),
  { role: 'user', content: task },
];
const taskIndex = messages.length - 1;
```

- 🧱 System prompt rebuilt for every task, so new memory is in front of the model.
- 📝 History the previous conversation, without its old system message.
- ✍️ New task the last message, and the one a hook may rewrite.

> There is no separate memory object inside the run. The message array is the short-term memory.
> **On screen:** The loop keeps no secret state. It keeps messages. The system prompt is rebuilt here, not carried in history, so memory written by the last task is already visible. A hook can rewrite the task at taskIndex before the model ever sees it.

05 · The loop in code

One for loop.

SRC/AGENT.JS · *source*

```
for (let step = 1; step <= maxSteps; step += 1) {
  const opening = await hook.run('PreModel', { step, maxSteps, messages, tools: TOOL_SCHEMAS });

  const beat = heartbeat(log, heartbeatMs);
  reply = await chatFn({ messages, tools: TOOL_SCHEMAS });
  beat.stop();

  const choice = reply?.choices?.[0] ?? {};
  const message = choice.message ?? {};
  // ...dispatch or answer...
}
```

The heartbeat is armed before the request and stopped on every path. A slow model must not look like a frozen program.

the design The loop is deliberately boring: one counter, one request, one dispatcher.

On screen: The loop is one for statement. The step budget is the first bound. Before the request, PreModel fires. Then the heartbeat starts, the model is asked, and the heartbeat stops. Nothing is hidden in a framework. You can read the whole control flow on one screen.

06 · Lifecycle

Nine places to get a turn.

`SessionStart` · `UserPromptSubmit` · `PreModel` · `PostModel` · `PreToolUse` · `PostToolUse` · `Stop` · `StepLimit` · `SessionEnd`

Before the first request — SessionStart and UserPromptSubmit can refuse or rewrite.

Every step — PreModel, PostModel, PreToolUse, PostToolUse.

At the end — Stop or StepLimit, then SessionEnd always.

> The hook calls cost nothing when no handlers are attached. That is why the loop can carry all nine.
> **On screen:** The nine hook names are the lifecycle. Each one is a call site in this file. Two fire before the first request, four inside every step, and three at the end. With no hooks attached, each call returns an empty decision and the loop carries on.

07 · Dispatch

Every call gets a result.

SRC/AGENT.JS · *source*

```
const pre = await hook.run('PreToolUse', { name, args: rawArgs, step });
const result = pre.blocked
  ? 'DENIED: a PreToolUse hook blocked this call. Try another approach.'
  : await runTool(name, rawArgs, ctx);

const post = await hook.run('PostToolUse', { name, result, step, ok });
messages.push({
  role: 'tool',
  tool_call_id: call.id,
  content: String(post.replace ?? result),
});
```

A refusal is still an observation. The model gets a matching tool message and a chance to choose another route.

the invariant A tool_call_id without a tool result makes the next request invalid. The loop never leaves one behind.

On screen: Dispatch is three decisions: may it run, what did it return, and what does the model read back? A PreToolUse block becomes a DENIED string, not an exception. PostToolUse may replace the observation. Whatever happens, every tool call gets exactly one tool message. That invariant keeps the next request valid.

08 · How it ends

Three ways a run stops.

Answer — No tool calls. Stop fires, then SessionEnd. The result is ok: true .

Step limit — The budget runs out. StepLimit fires, then SessionEnd.

Request failed — The model call threw. SessionEnd still fires.

> Failures get the same lifecycle as successes. SessionEnd is the finally block of the run.
> **On screen:** A run ends with an answer, with the step limit, or with a failed request. Stop is the happy path. StepLimit and request_failed are the failure paths. SessionEnd runs in every case. Cleanup, timing and audit logs have one reliable home.

09 · Recap

The loop is the program.

• 1 exported function

• 7 beats per step

• 9 hook moments

→

Injected dependencies. The loop never reaches for a global.

→

Messages are the state. System prompt, history, task, observations.

→

Every call gets a result. A tool_call_id never goes unanswered.

→

Endings have a lifecycle. Stop, StepLimit, request_failed, then SessionEnd.

> **next** Read agent.js first. Every other file exists to make one of its beats possible.
> **On screen:** One function, seven beats, nine hook moments. The loop injects its dependencies, keeps state in messages, and never leaves a tool call unanswered. Read agent.js first. The rest of src is the detail behind its beats.